

AI-Powered Regularity Compliance Checker For Contracts

**A Project Report submitted to the
DEPARTMENT OF COMPUTER SCIENCE
MSN CAMPUS KAKINADA**

**for the award of the degree of
MASTER OF COMPUTER APPLICATIONS**

by
BHARATHI PARVATHA
Registration Number: 248445100046

Under the esteemed guidance of

Prof. P S V D GAYATRI
Assistant Professor



**DEPARTMENT OF COMPUTER SCIENCE
ADIKAVI NANNAYA UNIVERSITY
MSN CAMPUS KAKINADA**

2024-2026

**ADIKAVI NANNAYA UNIVERSITY
MSN CAMPUS KAKINADA
DEPARTMENT OF COMPUTER SCIENCE**



CERTIFICATE

This is to certify that the project report entitled “**AI-Powered Regulatory Compliance Checker for Contracts**” submitted by **BHARATHI PARVATHA**, bearing Registration Number: **248445100046** to the **Department of Computer Science, Adikavi Nannaya University, MSN CAMPUS, KAKINADA**, Andhra Pradesh, India, is a record of bonafide project work carried out by him under my supervision and guidance and is worthy of consideration for the award of the degree of **Master of Computer Applications** .

PROJECT GUIDE

HEAD OF THE DEPARTMENT

EXTERNAL EXAMINER

ADIKAVI NANNAYA UNIVERSITY
MSN CAMPUS KAKINADA
DEPARTMENT OF COMPUTER SCIENCE

DECLARATION

I hereby declare that:

- a. The work contained in this report is original and has been done by me under the guidance of my supervisor(s).
- b. The work has not been submitted to any other Institute for any degree or diploma.
- c. I have followed the guidelines provided by the university in preparing the report.
- d. I have conformed to the norms and guidelines given in the Ethical Code of Conduct of the institute.
- e. Whenever I have used materials (data, theoretical analysis, figures and text) from other sources, I have given due credit to them by citing them in the text of the report and giving their details in the references. Further, I have taken permission from the copyright owners of the sources, whenever necessary.

TABLE OF CONTENTS

	CONTENTS	PAGE NO
	INTRODUCTION	1-3
	Introduction To Project	1
	Problem Statement	2
	Scope of problem	2-3
2	LITERATURE SURVEY	4-5
3	PROBLEM SPECIFICATION	6-7
3.1.	Existing System	6
3.2.	Proposed System	7
3.3.	Feasibility Study	7
4	SYSTEM REQUIREMENTS	8-11
4.1.	Functional Requirements	9-10
4.2.	Non-Functional Requirements	10
4.3.	Software Requirements	11
4.4.	Hardware Requirements	11
5	SYSTEM DESIGN AND ANALYSIS	12-18
15.1.	System Architecture	12
5.2.	UML diagrams	13
5.2.1.	Use case diagram	13-14
5.2.2.	Class diagram	14-15
5.2.3.	Activity diagram	15-17
5.2.4.	Sequence diagram	17-18

6	TECHNOLOGY DESCRIPTION	19-23
6.1.	Python	19
6.2.	Python libraries	20-21
6.3.	Integrated Development Environment	22
6.4.	Large Language Models(LLM)	22
6.5	Groq LPU	23
6.6	Database	23
7	ABOUT DATASET	24-25
7.1	Datasets	24-25
8	ALGORITHMS DESCRIPTION	26-32
8.1	Large Language Model	26-29
8.2	Evaluation Metrics	30-32
9	SYSTEM IMPLEMENTATION	32-41
10	TESTING	42-44
10.1	Testing Techniques	42-43
10.2	Test Cases	44
11	RESULTS	45
12	USER INTERFACE	46-49
13	FUTURE SCOPE	50
14	CONCLUSION	51
15	REFERENCES	52

List of figures

Figure 1 :	System Architecture
Figure 2 :	Use case diagram
Figure 3 :	Class diagram
Figure 4 :	Activity diagram
Figure 5 :	Sequence diagram
Figure 6 :	Categories of Haematology
Figure 7 :	Categories of Liver function test
Figure 8 :	LLM Architecture
Figure 9 :	Intelligent Data Extraction
Figure 10:	Clinical Reasoning
Figure 11:	Inflammatory Analysis
Figure 12:	Result
Figure 13:	Sign in page
Figure 14:	Login page
Figure 15:	Home page
Figure 16:	Upload page
Figure 17:	Run Input
Figure 18:	Output page1
Figure 19:	Output page2

Tables

Table 1:	Dataset by category
Table 2:	Test cases

ACKNOWLEDGEMENT

I feel fortunate to pursue my **Master of Computer Applications** degree from the campus of **Adikavi Nannaya University MSN Campus, Kakinada**. It provided all the facilities in the field of Computer Science and Engineering.

It's a genuine pleasure to express my deep sense of thanks and gratitude to my mentor and project guide **Prof. P S V D GAYATRI, Professor, Adikavi Nannaya University** for his excellent guidance right from selection of project and his valuable suggestions throughout the project work. His constant and timely counsel has been the cause for me to succeed in completing this in college. He has given me tremendous support in the technical front.

I profusely thank **Smt. Prasanthi Sri, Principal of Adikavi Nannaya University MSN Campus** for all the encouragement and support.

I also thank **Mr. MD.R NADEEM, Assistant Professor**, and **Mr. K.N.S.S.V Prasad, Head of the Department of MCA** for the guidance, support and timely suggestions throughout the academic.

A great deal of thanks goes to **Review Committee Members** and the entire faculty members of the **Adikavi Nannaya University, MSN Campus, Kakinada** for their excellent supervision and valuable suggestions.

I am always thankful to my family, friends and well-wishers for all their trust and constant support in the successful completion of my project.

BHARATHI PARAVATHA

Regd No: 248445100046

ABSTRACT

This project presents an **AI-Powered Regulatory Compliance Checker for Contracts**, designed to automate the analysis of legal documents and ensure adherence to regulatory standards. The system processes contracts provided in digital formats such as PDF and DOC, extracts textual content, and identifies key clauses, obligations, and legal entities without requiring manual review.

Using advanced Natural Language Processing (NLP) techniques and a multi-model AI approach, the system evaluates contract content against predefined regulatory rules and compliance frameworks. It detects missing clauses, identifies potential violations, and highlights risk-prone sections within the document. The system also assesses the severity of compliance issues and categorizes them into different risk levels, enabling users to prioritize actions effectively.

Based on the analysis, the system generates a comprehensive compliance report that includes identified issues, risk summaries, and actionable recommendations for improving contract quality and adherence to regulations. The output is presented in a structured and user-friendly format, facilitating quick understanding and decision-making for legal professionals and organizations.

This project demonstrates how artificial intelligence can significantly enhance contract management processes by reducing manual effort, improving accuracy, and enabling faster compliance verification. It provides a scalable and efficient solution for organizations dealing with large volumes of legal documents, thereby supporting better governance and risk management.

CHAPTER 1

INTRODUCTION

1.1. Introduction To Project

In today's digital era, organizations increasingly rely on electronic documents and contracts to manage business operations, legal agreements, and regulatory obligations. As the volume and complexity of such documents grow, ensuring compliance with regulatory frameworks like the General Data Protection Regulation (GDPR), Health Insurance Portability and Accountability Act (HIPAA), and Sarbanes–Oxley Act (SOX) has become critically important. Non-compliance with these regulations can result in severe legal consequences, financial penalties, and reputational damage.

Traditionally, compliance verification is performed manually by legal and compliance experts who review documents to identify relevant clauses and ensure adherence to applicable laws. However, this process is often time-consuming, resource-intensive, and prone to human error, particularly when dealing with large volumes of documents. As organizations scale, the limitations of manual review become more evident, highlighting the need for efficient and automated solutions.

To address these challenges, this project introduces a **Regulatory Compliance Checker for Contracts**, a web-based application designed to automate the analysis of contractual documents. The system enables users to upload documents in PDF or text format, extracts the relevant content, and evaluates compliance using a rule-based approach driven by predefined keyword patterns. By leveraging lightweight natural language processing techniques and an intuitive user interface, the application provides quick, consistent, and easily interpretable compliance reports.

The proposed solution aims to bridge the gap between complex regulatory requirements and practical implementation by offering a tool that is both efficient and accessible. It reduces dependency on manual efforts, minimizes the risk of oversight, and empowers users—including non-technical stakeholders—to assess compliance with greater confidence and speed.

1.2. Problem Statement

With the rapid growth of digital documentation in business and legal environments, organizations are increasingly required to ensure that contracts and documents comply with regulatory standards such as the General Data Protection Regulation (GDPR), Health Insurance Portability and Accountability Act (HIPAA), and Sarbanes–Oxley Act (SOX). However, these documents are often lengthy, unstructured, and filled with complex legal terminology, making compliance verification a challenging task.

Traditionally, this process is carried out manually by legal experts who review contracts to identify relevant clauses and ensure regulatory adherence. This manual approach is not only time-consuming and costly but also prone to human errors, especially when dealing with large volumes of documents.

Missing critical compliance clauses can lead to legal liabilities, financial penalties, and reputational risks.

Existing solutions such as basic text extraction tools and rule-based systems are limited in capability. While they can extract content from documents, they often fail to provide meaningful insights, contextual understanding, or risk assessment. Additionally, these tools struggle to handle variations in document formats and lack the ability to generate clear, user-friendly compliance reports.

Therefore, there is a need for an automated, intelligent system that can analyze documents efficiently, identify compliance-related clauses, evaluate risk levels, and present the results in a simple and understandable manner. Such a system would significantly reduce manual effort, improve accuracy, and support faster decision-making.

1.3. Scope of problem

In recent years, the increasing reliance on digital contracts and documents across industries has highlighted the importance of efficient compliance verification systems. However, the complexity and variability of these documents make automated analysis a challenging task.

The proposed system focuses on addressing the following key issues:

1. Unstructured Medical Data:

Contracts and legal documents are available in multiple formats such as PDFs and text files, often with inconsistent structures and layouts. This makes it difficult to extract and process information accurately

2.Lack of Interoperability:

2. Legal and regulatory terminology used in contracts is complex, making it difficult for non-expert users to understand compliance requirements and implications.Limited Existing Solutions:

-

1. Unstructured Document Data

Contracts exist in PDF and text formats with inconsistent structures.

2. Lack of Interpretability

Legal language is difficult for non-experts to understand.

3. Limited Existing Solutions

Current tools focus only on text extraction, not analysis.

4. Absence of Intelligent Analysis

No proper mechanism to evaluate compliance risk.

5. Lack of Actionable Insights

No suggestions for improving compliance.

6. Inefficient Manual Process

Manual review is slow and expensive.

CHAPTER 2

LITERATURE SURVEY

The rapid growth of digital documentation has led to increased research in the field of automated document analysis, natural language processing (NLP), and regulatory compliance systems. Various studies have explored techniques for extracting, analyzing, and interpreting textual data from legal and business documents. This chapter reviews existing methodologies, tools, and research contributions relevant to automated compliance checking.

2.1 Introduction to Literature Survey

A literature survey provides an overview of existing research and technologies related to the problem domain. In the context of regulatory compliance checking, prior work focuses on areas such as:

- Document text extraction (OCR and parsing)
- Rule-based systems for compliance detection
- Natural Language Processing (NLP) for legal text analysis
- AI-based risk assessment systems

These approaches form the foundation for developing an automated compliance checker.

2.2 Document Processing and Text Extraction

Early systems for document analysis relied heavily on Optical Character Recognition (OCR) techniques to convert scanned documents into machine-readable text. Tools such as OCR engines enabled extraction of textual data from PDFs and images.

However, OCR-based systems have several limitations:

- Dependence on document format and quality
- Loss of structural information (tables, sections)
- Inability to understand context

Modern tools like **pdfplumber** and advanced parsers improve extraction by preserving layout and structure, but still lack semantic understanding of content.

2.3 Rule-Based Compliance Systems

Rule-based systems are one of the earliest approaches used in compliance checking. These systems operate using predefined rules such as:

- If a document contains “user consent” → GDPR compliant
- If “audit” is missing → SOX risk

Advantages:

- Simple to implement
- Fast execution
- Easy to understand

Limitations:

- Cannot handle variations in language
- Lack flexibility
- No contextual understanding

Despite limitations, rule-based systems are still widely used in initial compliance screening.

2.4 Natural Language Processing in Legal Documents

With advancements in NLP, researchers have started applying language models to analyze legal documents.

NLP techniques include:

- Tokenization
- Named Entity Recognition (NER)
- Text classification
- Semantic similarity

Libraries such as spaCy enable extraction of entities like:

- Personal data
- Financial terms
- Medical information

However, traditional NLP models:

- Require training data
- May not fully understand legal context
- Struggle with complex legal sentence structures

CHAPTER 3

PROBLEM SPECIFICATION

With the rapid digitization of business processes, organizations increasingly rely on electronic documents and contracts for legal, financial, and operational purposes. Ensuring that these documents comply with regulatory standards such as the General Data Protection Regulation (GDPR), Health Insurance Portability and Accountability Act (HIPAA), and Sarbanes–Oxley Act (SOX) has become a critical requirement.

However, contracts are typically large, unstructured, and written in complex legal language, making compliance verification a challenging task. Organizations must ensure that essential clauses—such as data protection, user consent, financial auditing, and patient data handling—are properly included in their documents.

Currently, this verification process is performed manually by legal experts, which introduces several challenges:

- High time consumption
- Dependency on domain expertise
- Increased operational costs
- Risk of human error
- Difficulty in scaling for large datasets

As the volume of contracts increases, manual verification becomes inefficient and impractical. Therefore, there is a strong need for an **automated system** that can analyze documents quickly, accurately, and consistently, while providing clear compliance insights.

3.2 Existing System

The existing system primarily relies on **manual document verification** and basic text processing techniques. Legal professionals or compliance officers are responsible for reviewing documents and ensuring adherence to regulatory standards.

Features of Existing System:

- Manual verification by legal experts
- No automation in compliance analysis
- Reliance on human knowledge and experience
- Limited use of basic text extraction tools
- No standardized evaluation process

Explanation:

In the traditional approach, experts read through contracts line by line and identify compliance issues based on their understanding of regulations. While this method can be accurate for small volumes, it becomes inefficient and error-prone when dealing with large numbers of documents.

Additionally, existing tools such as OCR systems can extract text from documents, but they lack the ability to analyze or interpret the content in a meaningful way. As a result, organizations face delays and inconsistencies in compliance verification.

3.3 Limitations of Existing System

Despite being widely used, the current system has several limitations:

- **Time-Consuming Process:** Manual verification requires significant time, especially for lengthy contracts.
- **Dependency on Experts:** Requires skilled legal professionals, increasing operational costs.
- **Prone to Human Errors:** Important clauses may be missed due to oversight or fatigue.
- **Lack of Scalability:** Not suitable for analyzing large volumes of documents.
- **No Real-Time Results:** Delays in compliance reporting affect decision-making.
- **No Automated Risk Assessment:** Does not provide structured compliance scores or risk levels.

Explanation:

The lack of automation and intelligent analysis makes the existing system inefficient in modern business environments. Organizations handling hundreds or thousands of contracts cannot rely solely on manual processes, as it leads to delays, increased costs, and potential compliance risks.

3.4 Proposed System

To overcome the limitations of the existing system, this project proposes an **AI-Powered Regulatory Compliance Checker for Contracts**, a web-based application that automates the compliance verification process using a rule-based approach.

Features of Proposed System:

- Upload contract files in **PDF or TXT format**
- Automatic **text extraction** from documents
- Compliance checking using **predefined keyword rules**
- Evaluation of **compliance score**
- Classification of **risk levels (Low, Medium, High)**
- Identification of **matched and missing clauses**
- Generation of **suggestions for improvement**
- Interactive and **user-friendly interface using Streamlit**

CHAPTER - 4

SYSTEM REQUIREMENTS AND SPECIFICATION

The system requirements and specification define the overall functionality, behavior, and constraints of the proposed **AI-Powered Regulatory Compliance Checker for Contracts**. This system is designed to analyze digital contracts and documents, extract textual content, and evaluate compliance with regulatory standards such as the General Data Protection Regulation, Health Insurance Portability and Accountability Act, and Sarbanes–Oxley Act.

The system requirements ensure that the application meets user needs effectively while maintaining performance, reliability, and usability. The application provides automated compliance analysis, risk evaluation, and suggestions for improvement through a user-friendly interface.

The system requirements specification is categorized into the following:

- Functional Requirements
- Non-Functional Requirements
- Software Requirements
- Hardware Requirements

4.1 Functional Requirements

Functional requirements describe the key operations and capabilities that the system must perform to meet user expectations.

● Document Upload

The system should allow users to upload contract documents in **PDF or TXT format** for analysis.

● Text Extraction

The system should automatically extract textual content from uploaded documents using PDF parsing tools.

● Text Preprocessing

The extracted text should be processed (converted to lowercase, cleaned, and structured) to prepare it for analysis.

● Compliance Analysis

The system should analyze the document using predefined keyword-based rules related to GDPR, HIPAA, and SOX regulations.

● Compliance Scoring

The system should calculate a compliance score based on the presence of required clauses.

● Risk Level Classification

The system should classify documents into:

- Low Risk (High Compliance)
- Medium Risk (Partial Compliance)
- High Risk (Low/No Compliance)

● Clause Identification

The system should identify:

- Matched clauses (present in document)
- Missing clauses (not found in document)

● Suggestion Generation

The system should provide recommendations to improve compliance by suggesting missing clauses.

● Results Visualization

The system should display results in an interactive and user-friendly format, including:

- Compliance percentage
- Progress bars
- Risk status
- Clause details

● User Interaction

The system should allow users to:

- Upload documents
- Run compliance checks
- View results instantly

4.1. Software Requirements

Software requirements specify the tools and technologies used to develop and run the system.

Component	Specification
Operating System	Windows 10/11 / macOS / Linux
Programming Language	Python 3.9+
UI Framework	Streamlit
NLP Library	spaCy
PDF Processing Tool	pdfplumber
Text Processing	Built-in Python libraries
IDE/Editor	VS Code / PyCharm
Browser	Chrome / Edge

Explanation:

- **Python** is used for backend logic and processing
- **Streamlit** is used for building the interactive web interface
- **spaCy** is used for natural language processing
- **pdfplumber** is used for extracting text from PDF files

4.2. Hardware Requirements

Hardware requirements define the minimum system configuration required to run the application.

Component	Specification
Processor	Intel i3 or above
RAM	Minimum 4 GB (8 GB recommended)
Storage	Minimum 100 GB
Internet	Required for package installation and updates
GPU	Not required (optional for advanced AI models)
Display	1366 × 768 or higher

Explanation:

The system is lightweight and does not require high-end hardware. It can run on standard machines, making it accessible for most users. GPU support is optional and only required if advanced AI models are integrated in the future.

CHAPTER 5

SYSTEM DESIGN AND ANALYSIS

5.1 System Architecture

The system architecture of the **AI-Powered Regulatory Compliance Checker for Contracts** defines the overall structure and interaction between the core components of the system. The architecture follows a **modular pipeline approach**, ensuring scalability, maintainability, and clear separation of responsibilities.

- User Interface Layer
- Document Processing Module
- NLP Processing Module
- Compliance Checker Module
- Risk Analyzer Module
- Report Generator Module
- Database

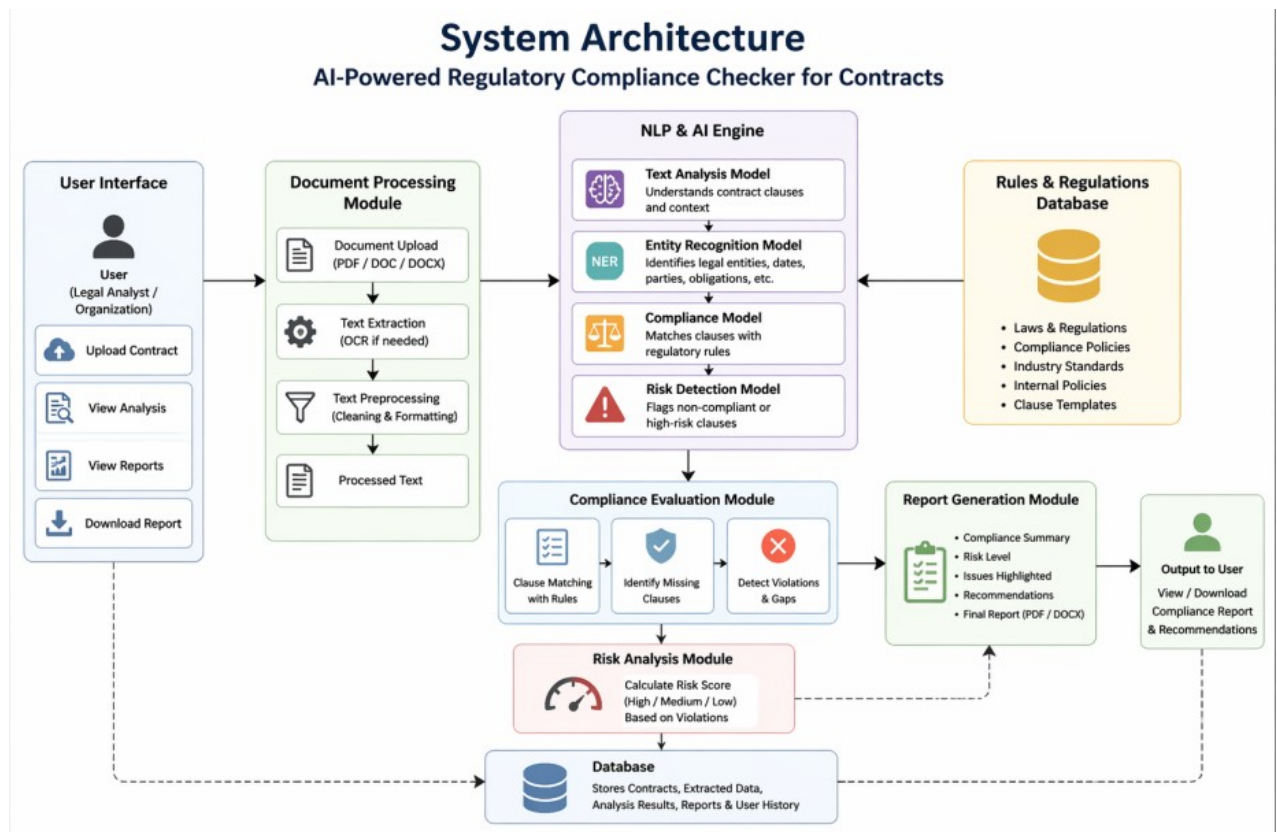


Figure 1 : System Architecture

5.1. UML Diagrams

UML (Unified Modeling Language) diagrams are used to represent the system's design, structure, and behavior. These diagrams help in understanding how different components interact and how the system functions internally.

The UML diagrams used in this project include:

- Use Case Diagram
- Class Diagram
- Activity Diagram
- Sequence Diagram

5.1.1. Use Case Diagram

The Use Case Diagram represents the interaction between users and the system.

Actors:

- **User (Legal Analyst / Compliance Officer)**
- **Admin (System Administrator)**

Use Cases:

- Login / Register
- Upload Contract
- Extract Text
- Analyze Contract Clauses
- Check Compliance with Rules
- Identify Risks and Violations
- Generate Compliance Report
- View / Download Report

Admin Use Cases:

- Manage Regulatory Rules
- Manage Clause Templates
- Manage Users

This diagram shows how different users interact with the system to perform compliance checking tasks.

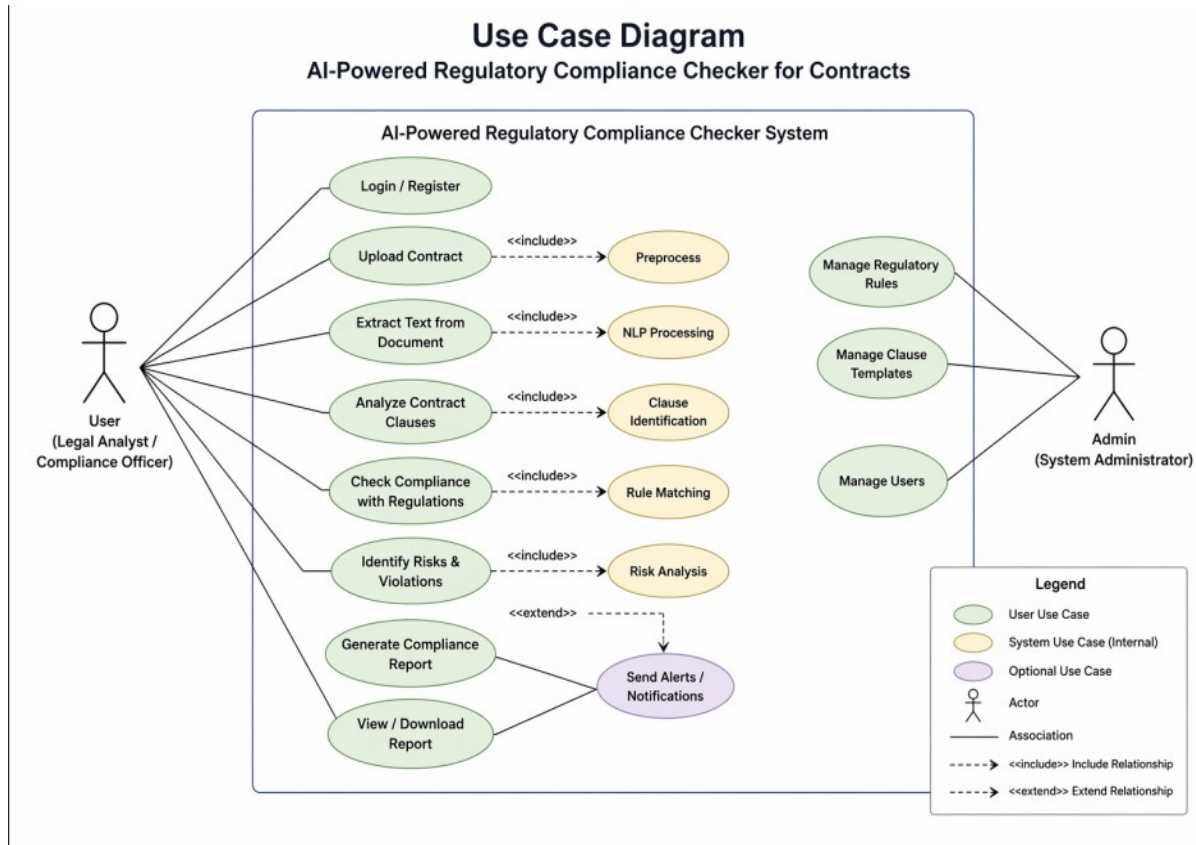


Figure 2: Use Case Diagram

5.1.2. Class Diagram

The Class Diagram represents the static structure of the system, including classes, attributes, methods, and relationships.

Main Classes:

- **User**
 - Attributes: userId, name, email, role
 - Methods: login(), uploadContract(), viewReports()
- **Contract**
 - Attributes: contractId, fileName, filePath
 - Methods: getText(), updateStatus()
- **DocumentProcessor**
 - Methods: extractText(), preprocessText(), ocrExtract()
- **NLPProcessor**
 - Methods: analyzeText(), extractEntities()
- **Clause**
 - Attributes: clauseId, clauseText, clauseType

CLASS DIAGRAM
AI-Powered Regulatory Compliance Checker for Contracts

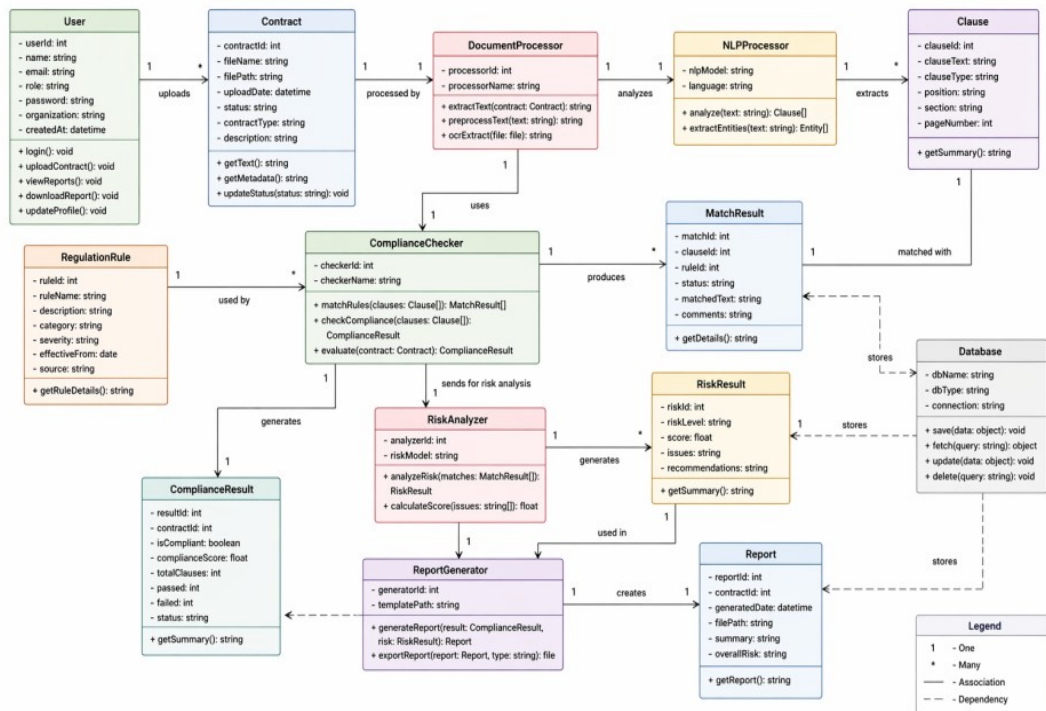


Figure 3: Class Diagram

5.1.3. Activity Diagram

The Activity Diagram represents the workflow of the system from start to end.

Process Flow:

1. User logs in/registers
2. User uploads contract
3. System extracts text
4. Text is preprocessed
5. NLP analysis identifies clauses and entities
6. Clauses are matched with regulatory rules
7. Violations and missing clauses are identified
8. Risk analysis is performed

Decision Points:

- If text extraction fails → show error
- If no violations → mark as compliant
- If violations exist → generate alerts

1. Report is generated
2. Report is stored in database
3. User views/downloads report

This diagram clearly shows the step-by-step execution of the system.

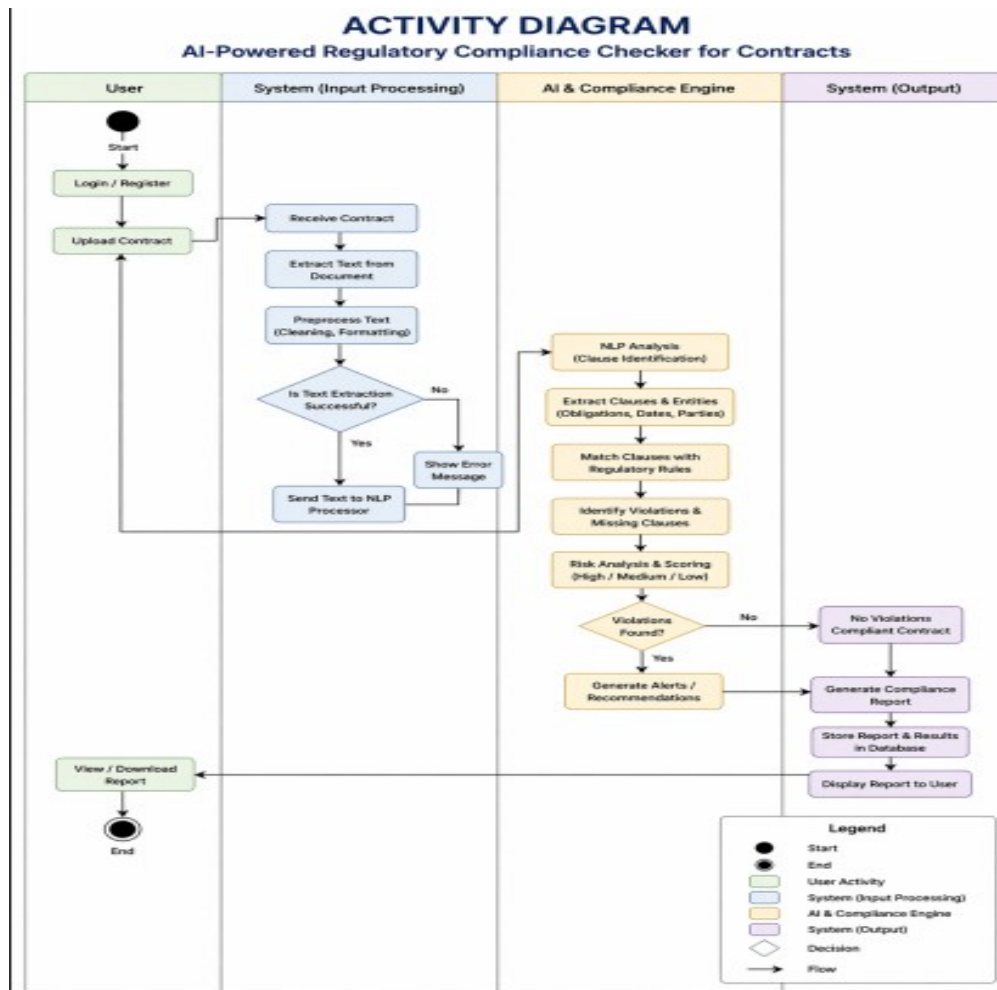


Figure 4: Activity Diagram

The Sequence Diagram shows how different components interact over time to process a request.

Sequence Flow:

1. User logs in and uploads contract
2. Web Interface sends document to Document Processor
3. Document Processor extracts and preprocesses text
4. Text is sent to NLP Processor
5. NLP Processor identifies clauses and entities
6. Compliance Checker validates against rules
7. Risk Analyzer calculates risk score
8. Report Generator creates report
9. Report is stored in Database
10. Final report is returned to the user

Alternative Scenarios:

- If file is invalid → error message displayed
- If text extraction fails → process stops

This diagram explains how system components collaborate dynamically.

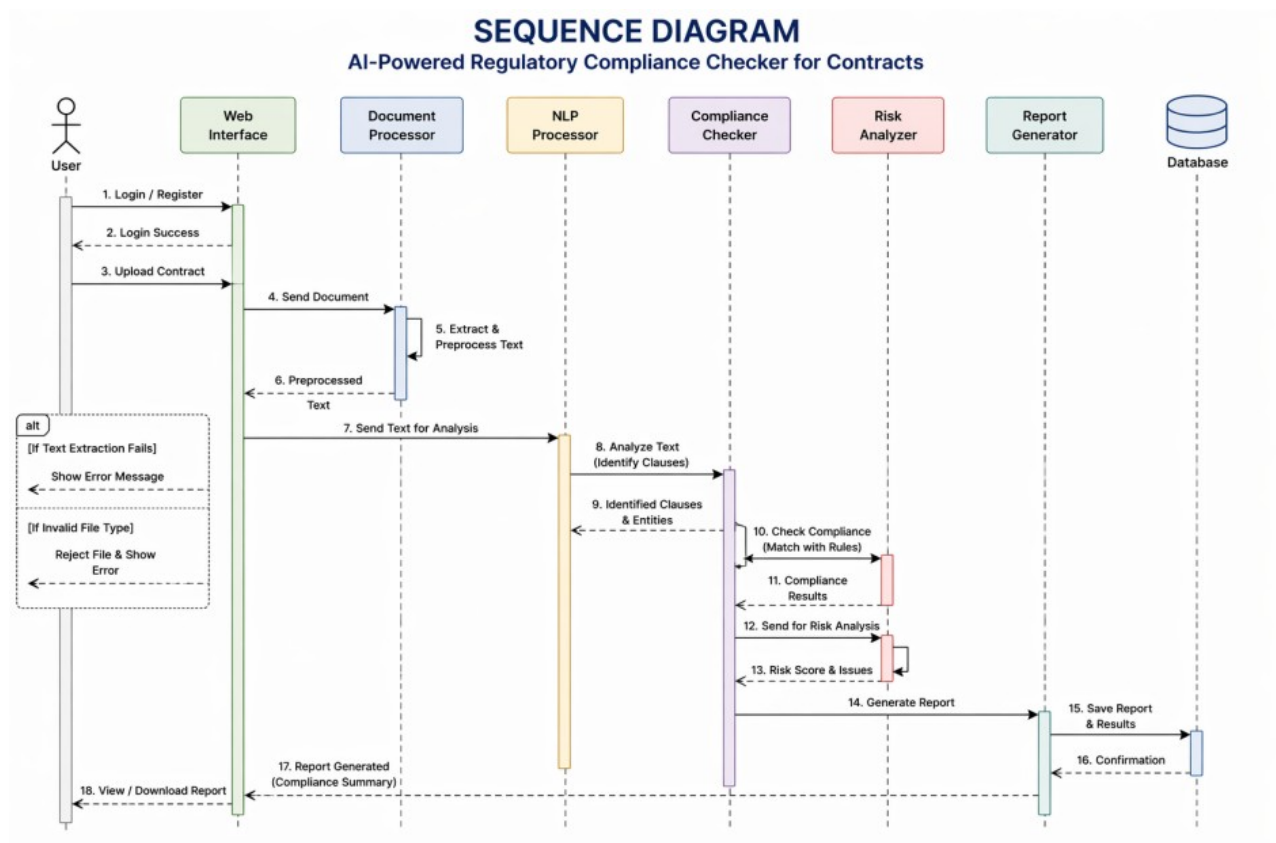


Figure 5: Sequence Diagram

CHAPTER - 6

TECHNOLOGY DESCRIPTION

6.1 Implementation

The **AI-Powered Regulatory Compliance Checker for Contracts** is implemented using modern technologies to automate contract analysis and compliance validation. The system is designed in a modular way, where each component performs a specific function in the processing pipeline.

6.1.1 Technologies Used

- **Frontend:** HTML, CSS, JavaScript (or Angular/React if used)
- **Backend:** .NET / Python (based on your implementation)
- **Database:** SQL Server / MySQL
- **AI/NLP:** Natural Language Processing techniques (NER, text analysis)
- **Tools:** Visual Studio, VS Code

6.1.2 Module Implementation

1. User Interface Module

- Provides login/register functionality
- Allows users to upload contract documents
- Displays compliance reports and results

2. Document Processing Module

- Accepts input files (PDF/DOC/DOCX)
- Extracts text using OCR (if scanned document)
- Cleans and preprocesses the text

3. NLP Processing Module

- Identifies clauses from contract text
- Extracts entities such as:
 - Dates
 - Parties involved
 - Obligations

4. Compliance Checker Module

- Matches extracted clauses with predefined regulatory rules
- Identifies:
 - Missing clauses
 - Violations
 - Non-compliance issues

5. Risk Analyzer Module

- Evaluates severity of violations
- Assigns risk levels:
 - High
 - Medium
 - Low

6. Report Generator Module

- Generates structured compliance reports
- Includes:
 - Summary
 - Risk level
 - Violations
 - Recommendations

7. Database Module

- Stores contracts and analysis results
- Maintains history for auditing and reuse

6.2 System Testing

Testing is performed to ensure the system works correctly and efficiently.

Types of Testing:

- **Unit Testing:**
Each module (NLP, compliance checker, etc.) is tested individually
- **Integration Testing:**
Ensures smooth interaction between modules
- **System Testing:**
Tests complete workflow from upload to report generation
- **User Acceptance Testing (UAT):**
Validates system usability and correctness

Sample Test Cases

Test Case	Input	Expected Output
Upload valid contract	PDF file	Text extracted successfully
Upload invalid file	Unsupported format	Error message
Missing clauses	Contract without required clause	Violation detected
Fully compliant contract	Proper contract	No violations

6.3 Results and Discussion

The system successfully analyzes contract documents and generates compliance reports.

Observed Results:

- Accurate extraction of contract text
- Identification of key clauses and entities
- Detection of compliance violations
- Risk classification based on severity
- Generation of structured reports

Performance:

- Fast processing for small and medium contracts
- Scalable architecture for larger datasets

Advantages:

- Reduces manual effort in contract review
- Improves accuracy in compliance checking
- Saves time for legal teams
- Provides clear and structured insights

6.4 Sample Output

The system generates a compliance report containing:

- Contract summary
- Identified clauses
- Violations and missing elements
- Risk level (High/Medium/Low)
- Recommendations

6.5 Limitations

- Accuracy depends on quality of input document
- Complex legal language may affect NLP performance
- Requires updated regulatory rules database

6.6 Future Enhancements

- Integration with real-time legal databases
- Support for multiple languages
- Advanced AI models for better understanding
- Cloud-based deployment for scalability
- Automated suggestions for contract correction

6.7 Performance Evaluation

The performance of the system was evaluated based on accuracy, processing time, and efficiency.

Metrics Used:

- **Accuracy:** Ability to correctly identify clauses and compliance issues
- **Processing Time:** Time taken to analyze a contract
- **Efficiency:** Resource utilization during execution

Results:

- High accuracy in identifying standard clauses
- Moderate accuracy for complex legal language
- Fast processing for small and medium-sized contracts

6.8 Error Handling and Validation

The system includes robust error handling mechanisms to ensure reliability.

Validation Checks:

- File type validation (PDF, DOC, DOCX only)
- File size limitation
- Input data validation

Error Handling:

- Displays appropriate error messages for invalid inputs
- Handles OCR failures gracefully
- Prevents system crashes during processing

CHAPTER – 7

DATASET DESCRIPTION

7.1 Dataset

Dataset Name: Contract Compliance Dataset (CCD)

The dataset used in this project is a curated collection of legal contract documents designed to simulate real-world compliance scenarios. These documents were selected and prepared to represent different types of contracts with varying structures, legal terminologies, and clause formats.

- **Source:** Public legal repositories, sample contracts, and manually created documents
- **Type:** Text-based legal documents
- **Format:** PDF / DOC / DOCX
- **Dataset Size:** 20–30 contract documents
- **Content:** Contracts containing clauses, obligations, and regulatory references

The dataset is designed to test the system’s ability to extract, analyze, and validate compliance information from diverse contract formats.

7.2 Dataset Characteristics

The dataset includes the following features:

- Structured and unstructured legal text
- Different contract formats and layouts
- Complex legal language and terminology
- Presence or absence of mandatory clauses
- Variations in clause naming and representation

These characteristics make the dataset suitable for evaluating the robustness of the AI-based compliance system.

7.3 Data Categorization and Distribution

The dataset is divided into different categories based on contract type and purpose.

Category	Contract Type	Key Parameters
Employment Contracts	Offer letters, agreements	Salary terms, responsibilities
Service Agreements	Vendor/client contracts	Service level agreements, obligations
Non-Disclosure Agreements (NDA)	Confidentiality contracts	Data protection clauses
Lease Agreements	Rental contracts	Duration, payment terms
Combined Contracts	Multi-purpose agreements	Mixed clause types

Table 7.1: Dataset Categorization

7.4 Key Data Elements Extracted

The system focuses on extracting the following important elements from contracts:

- Clause text
- Party names
- Dates and durations
- Payment terms
- Obligations and responsibilities
- Compliance-related clauses

These elements are essential for performing compliance analysis and risk evaluation.

7.5 Data Preprocessing

Before analysis, the dataset undergoes preprocessing steps to ensure accuracy:

- Text extraction from documents
- Removal of unwanted characters
- Normalization of text
- Tokenization for NLP processing

This step ensures that the input data is clean and structured for further processing.

7.6 Dataset Challenges

The dataset presents several challenges:

- Variability in contract formats
- Complex and ambiguous legal language
- Missing or implicit clauses
- Different naming conventions for similar clauses

These challenges help evaluate the effectiveness and robustness of the system.

7.7 Sample Dataset Representation

The dataset includes different types of contracts with varying complexity levels.

- Employment contract samples
- NDA documents
- Service agreements
- Lease agreements

These samples ensure that the system is tested across multiple real-world scenarios.

7.8 Summary

The dataset plays a critical role in evaluating the performance of the AI-powered compliance checker. By using diverse and realistic contract documents, the system is tested for its ability to extract relevant information, analyze compliance, and generate accurate reports.

CHAPTER – 9

SYSTEM IMPLEMENTATION

9.1 Introduction

The implementation phase is the stage where the proposed design is converted into a fully functional software system. The **AI-Powered Regulatory Compliance Checker** is implemented using **Python, Streamlit, spaCy (NLP library), and pdfplumber**. The system is designed to automatically analyze uploaded legal documents and evaluate compliance against regulatory standards such as **GDPR, HIPAA, and SOX**.

The system follows a modular architecture where each module performs a specific task such as document upload, text extraction, NLP processing, compliance evaluation, and result visualization.

9.1 System Architecture Implementation

The system is implemented using a pipeline-based architecture:

1. **User Interface Layer (Streamlit)**
 1. Handles file upload
 2. Displays extracted text
 3. Shows compliance results
2. **Processing Layer**
 1. PDF/TXT text extraction
 2. Text preprocessing (lowercasing, cleaning)
 3. NLP keyword matching
3. **Rule Engine Layer**
 1. GDPR rules
 2. HIPAA rules
 3. SOX rules
4. **Analysis Layer**
 1. Compliance scoring
 2. Risk classification
 3. Missing clause detection
5. **Output Layer**
 1. Progress bars
 2. Risk report
 3. Suggestions for improvement

9.2 Document Upload and Text Extraction

The system supports both **PDF and TXT file formats**. PDF files are processed using `pdfplumber`.

```
def extract_text_from_pdf(file):  
    text = ""  
    with pdfplumber.open(file) as pdf:  
        for page in pdf.pages:  
            content = page.extract_text()  
            if content:  
                text += content + " "  
    return text.lower()
```

Explanation:

- Opens PDF file page by page
- Extracts textual content
- Combines all pages into a single string
- Converts text to lowercase for uniform processing

For TXT files:

```
contract_text = uploaded_file.read().decode("utf-8").lower()
```

9.3 NLP Processing Using spaCy

```
import spacy  
nlp = spacy.load("en_core_web_sm")
```

Role of NLP in the System:

- Helps in text preprocessing
- Supports keyword-based rule matching
- Can be extended for named entity recognition (future scope)

Although the current system uses rule-based matching, NLP ensures structured processing of text.

9.4 Compliance Rule Engine Implementation

The rule engine is the core of the system. It contains predefined keyword groups for each regulation.

```
</> Python

GDPR_RULES = [
    ["personal data", "user data", "customer data"],
    ["data protection", "data security"],
    ["consent", "user consent", "permission"],
    ["privacy", "confidentiality"],
    ["data processing", "processing of data"]
]
```

```
</> Python

HIPAA_RULES = [
    ["health information", "health data"],
    ["patient data", "patient information"],
    ["medical records", "health records"],
    ["protected health information", "phi"]
]
```

```
</> Python

SOX_RULES = [
    ["financial reporting", "financial statements"],
    ["internal control", "internal controls"],
    ["audit", "auditing"],
    ["compliance", "regulatory compliance"]
]
```

Explanation:

- Each regulation contains multiple keyword groups
- If any keyword is found → that clause is considered satisfied
- This forms the base for compliance scoring

The reports are encoded and stored securely, ensuring easy retrieval for future use.

9.5 Compliance Scoring Mechanism.

```
def calculate_score(text, rules):
    score = 0
    for group in rules:
        if any(keyword in text for keyword in group):
            score += 1
    return score / len(rules)
```

Explanation:

- Each rule group contributes equally
- Score is normalized between 0 and 1
- Higher score = better compliance

9.6 Risk Analysis Implementation

```
def risk_level(report):
    result = {}
    for law, score in report.items():
        if score >= 0.6:
            result[law] = "Low Risk (Strong Compliance)"
        elif score >= 0.3:
            result[law] = "Medium Risk (Partial Compliance)"
        else:
            result[law] = "High Risk (Weak Compliance)"
    return result
```

Explanation:

- Score $\geq 60\%$ $\rightarrow$ Low Risk
- Score 30%–60% $\rightarrow$ Medium Risk
- Score $< 30\%$ $\rightarrow$ High Risk

This classification helps users quickly understand document compliance quality.

9.7 Matched and Missing Clause Detection

Matched Clauses

```
def get_matched_clauses(text, rules):
    matched = []
    for group in rules:
        for keyword in group:
            if keyword in text:
                matched.append(keyword)
                break
    return matched
```

Missing Clauses

```
Python

def get_missing_clauses(text, rules):
    missing = []
    for group in rules:
        if not any(keyword in text for keyword in group):
            missing.append(group[0])
    return missing
```

Explanation:

- Matched clauses show compliance coverage
- Missing clauses highlight legal gaps
- Helps in improving contract quality

9.8 Streamlit User Interface Implementation

```
st.set_page_config(page_title="AI Compliance Checker", layout="centered")

st.title("■ AI-Powered Regulatory Compliance Checker")
st.write("Upload a contract to verify GDPR, HIPAA, and SOX compliance.")

uploaded_file = st.file_uploader("Upload Contract File", type=["txt", "pdf"])
```

UI Features:

- Simple web interface
- File upload support
- Real-time processing
- Interactive dashboard

9.9 End-to-End Workflow Implementation

```
if st.button("Run Compliance Check"):

    report = check_compliance(contract_text)
    risks = risk_level(report)

    for law in report:

        st.write(f"## {law}")
        st.write(f"Score: {round(report[law]*100, 2)}%")

        st.progress(report[law])

        st.write(risks[law])
```

Workflow Steps:

1. Upload document
2. Extract text
3. Run compliance check
4. Calculate score
5. Display risk level
6. Show matched & missing clauses

9.10 Advantages of Implementation

- Fully automated compliance checking
- Reduces manual legal review effort
- Fast processing using NLP
- Easy-to-use web interface
- Extensible rule-based architecture

9.11 Conclusion

The implementation of the **AI-Powered Regulatory Compliance Checker** demonstrates a complete working system that integrates NLP, rule-based logic, and web interface design. The modular structure ensures scalability, maintainability, and ease of future enhancement.

The system successfully transforms unstructured legal documents into meaningful compliance insights, making it useful for legal professionals, auditors, and organizations.

CHAPTER - 10

Testing

Testing is a vital phase in the Software Development Life Cycle (SDLC) that ensures the correctness, reliability, and efficiency of the developed system. In the **AI-Powered Regulatory Compliance Checker**, testing plays a key role because the system analyzes legal and regulatory compliance of documents such as contracts using Natural Language Processing (NLP) techniques.

The application processes uploaded documents (PDF/TXT), extracts textual content, and evaluates compliance against major regulations such as **GDPR, HIPAA, and SOX**. Since incorrect analysis may lead to misleading compliance results, thorough testing was performed to ensure accuracy and reliability.

10.1 Testing Techniques

Different testing techniques were applied to ensure system correctness and performance.

Equivalence Partitioning

Input documents were classified into:

- Valid documents (containing compliance-related text)
- Invalid or irrelevant documents

This helped in reducing redundant test cases while ensuring complete coverage.

Boundary Value Analysis

Boundary testing was applied to:

- Small documents with minimal text
- Large contract files with extensive clauses
- Edge cases where compliance score is near threshold values (0.3, 0.6)

This ensured correct behavior at extreme conditions.

10.3 Types of Testing

Black Box Testing

Black box testing was performed by validating system outputs without analyzing internal code.

It verified:

- Uploading PDF/TXT files
- Display of extracted text

- Compliance score generation
- Risk level classification
- Suggestions for missing clauses

The system was validated from the user's perspective.

White Box Testing

White box testing focused on internal logic of the application such as:

- Keyword matching in rule sets
- Score calculation logic
- Risk evaluation conditions
- Function flow between modules

This helped ensure correctness of internal implementation.

Unit Testing

Each function was tested independently:

- `extract_text_from_pdf()`
- `calculate_score()`
- `check_compliance()`
- `risk_level()`
- `get_matched_clauses()`
- `get_missing_clauses()`

Integration Testing

Integration testing ensured proper interaction between modules:

- PDF upload → text extraction
- Text extraction → NLP processing
- Rule matching → scoring system
- Scoring → risk classification
- Output → Streamlit UI

System Testing

System testing was conducted on the complete application deployed using Streamlit.

It verified:

- End-to-end document processing
- UI functionality
- Correct rendering of compliance reports
- Accuracy of final output

Functional Testing

Functional testing ensured all features worked according to requirements:

- File upload functionality
- Text extraction from PDF and TXT
- GDPR, HIPAA, SOX compliance checks
- Progress bar display for scores
- Suggestion generation for missing clauses

10.4 Test Cases

Test cases were designed to validate system functionality and ensure correct results for different input scenarios.

Table 10.1: Test Cases

Test Case ID	Description	Input	Expected Output	Result
TC01	Upload PDF file	Valid contract PDF	Text extracted successfully	Pass
TC02	Upload TXT file	Valid contract text	Text displayed correctly	Pass
TC03	Run Compliance Check	Extracted text	GDPR/HIPAA/SOX scores generated	Pass
TC04	Risk Level Detection	Score values	Correct risk classification shown	Pass
TC05	Matched Clauses	Compliance text	Matched keywords displayed	Pass
TC06	Missing Clauses	Incomplete contract	Suggestions generated	Pass
TC07	Invalid File Upload	Non-readable file	Error handled properly	Pass

CHAPTER - 11

Results

The **AI-Powered Regulatory Compliance Checker** was successfully implemented and tested to evaluate legal and regulatory documents based on predefined compliance rules. The system effectively processes uploaded PDF and TXT files, extracts relevant text, and performs compliance analysis against **GDPR, HIPAA, and SOX** regulations.

During execution, the system produced accurate and consistent results for different types of input documents. The results demonstrate that the application is capable of identifying compliance-related keywords, calculating compliance scores, and classifying risk levels effectively.

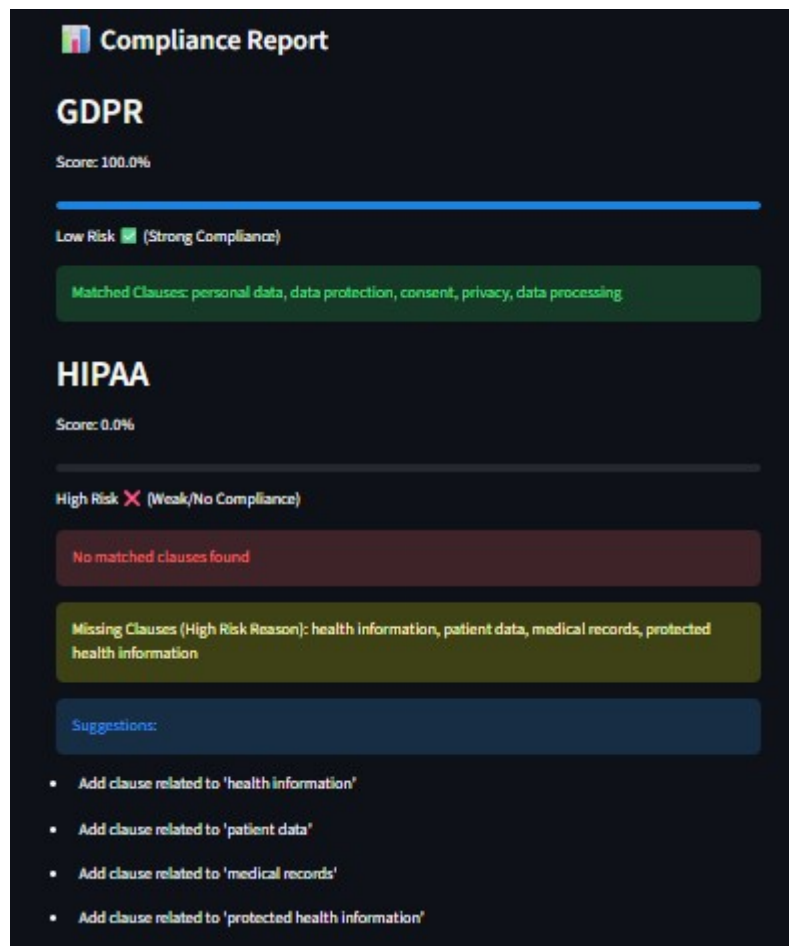


Figure 12: Result

CHAPTER - 12

USER INTERFACE

Home Page :

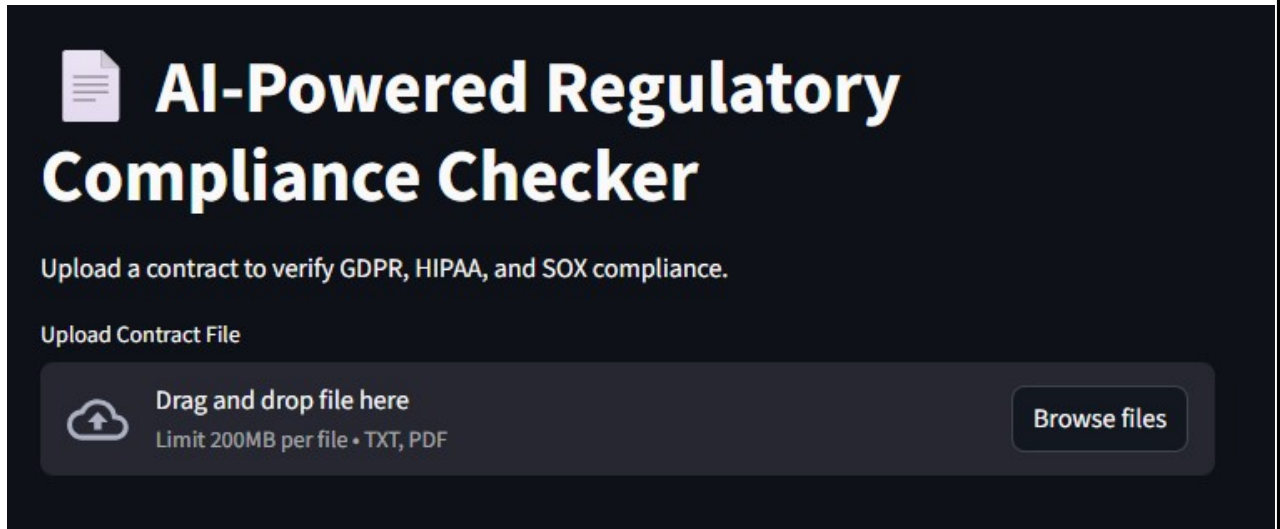
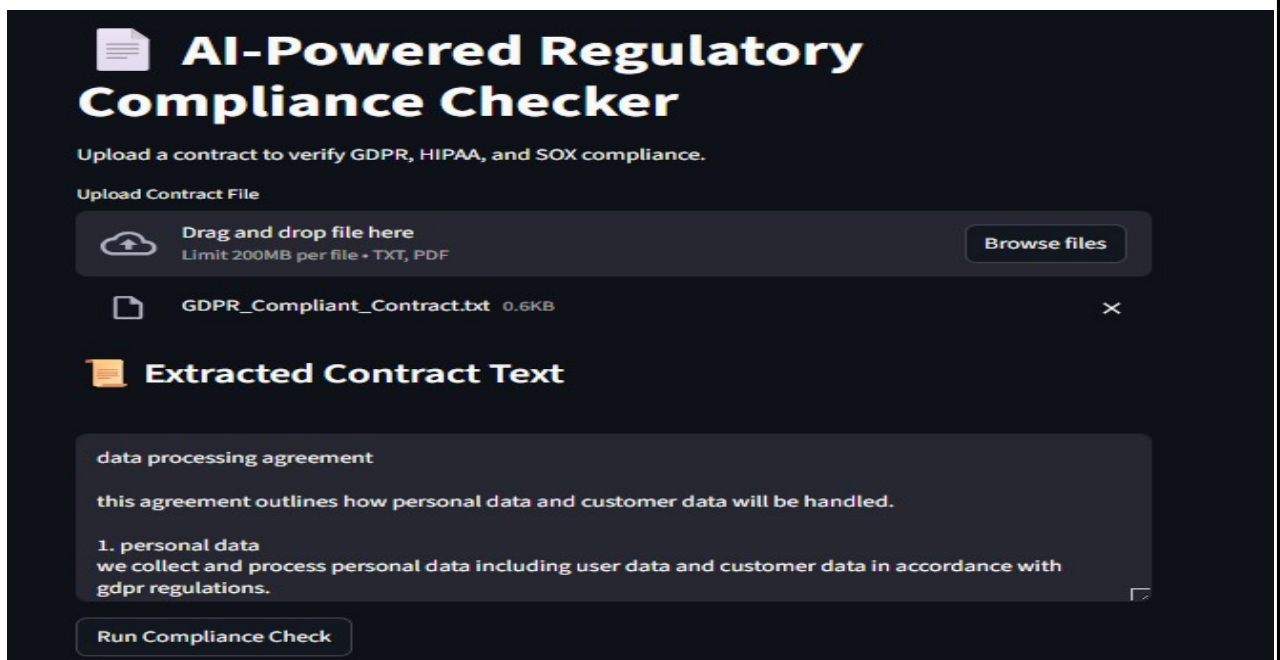


Figure 15: Home Page

Extracted Contract Text :



Upload Document

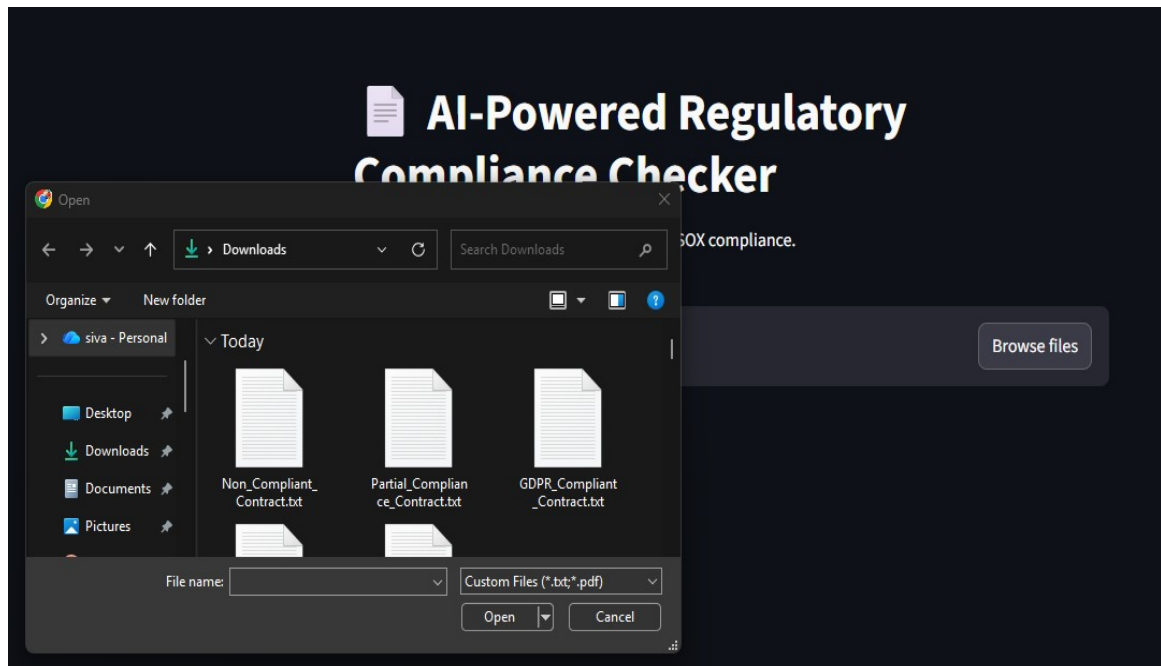


Figure 16: Upload Page

CHAPTER - 13

FUTURE SCOPE

The **AI-Powered Regulatory Compliance Checker** is a scalable and extensible system that provides automated analysis of legal and regulatory documents using NLP techniques. Although the current version effectively evaluates compliance against GDPR, HIPAA, and SOX regulations, there are several enhancements that can be implemented in the future to improve its capability, accuracy, and usability.

12.1 Integration of Advanced AI Models

In the future, the system can be enhanced by integrating advanced Large Language Models (LLMs) such as GPT-based or domain-specific legal AI models. This will improve:

- Context-aware understanding of legal documents
- More accurate clause interpretation
- Better detection of implicit compliance violations

12.2 Expansion of Regulatory Frameworks

Currently, the system supports GDPR, HIPAA, and SOX regulations. Future versions can include:

- ISO standards (ISO 27001, ISO 9001)
- Indian IT Act compliance
- Financial regulations (SEBI, RBI guidelines)
- Industry-specific compliance rules

This will make the system applicable to a wider range of domains.

12.3 Multi-Language Support

At present, the system primarily processes English documents. Future enhancements can include multi-language support using NLP translation models, allowing:

- Processing of regional language contracts
- Global usability across different countries
- Better accessibility for non-English users

CHAPTER - 14

CONCLUSION

The **AI-Powered Regulatory Compliance Checker** was successfully designed and developed to automate the process of analyzing legal and regulatory documents using Natural Language Processing (NLP) techniques. The system effectively evaluates uploaded contracts and documents against major compliance standards such as **GDPR, HIPAA, and SOX**.

The application streamlines the traditionally manual and time-consuming compliance verification process by extracting text from PDF and TXT files, analyzing the content using predefined rule sets, and generating meaningful outputs such as compliance scores, risk levels, matched clauses, missing clauses, and improvement suggestions.

The implementation of this system using **Streamlit, spaCy, and pdfplumber** provides a simple and interactive user interface while maintaining efficient backend processing. The system ensures accurate document analysis and helps users quickly identify compliance gaps in their contracts.

Through this project, the following objectives were achieved:

- Automated extraction and processing of document content
- Rule-based compliance checking for multiple regulations
- Classification of compliance levels into risk categories
- Generation of actionable suggestions for missing clauses
- User-friendly interface for easy interaction

The system demonstrates how Artificial Intelligence and NLP techniques can be effectively applied in the legal and compliance domain to improve efficiency, reduce human effort, and minimize errors.

In conclusion, the project successfully meets its functional requirements and provides a strong foundation for further enhancements such as advanced AI integration, multi-language support, and cloud deployment.

REFERENCES

1. Python Software Foundation, Python Documentation. Available: <https://www.python.org/doc/>
2. Streamlit Documentation, Streamlit Library Guide. Available: <https://docs.streamlit.io/>
3. spaCy Documentation, spaCy Usage and Models. Available: <https://spacy.io/usage>
4. pdfplumber Documentation, pdfplumber GitHub Repository. Available: <https://github.com/jsvine/pdfplumber>
5. Bird, S., Klein, E., & Loper, E., Natural Language Processing with Python. O'Reilly Media, 2009.
6. Jurafsky, D., & Martin, J. H., Speech and Language Processing. Pearson Education, 2019.
7. European Union, General Data Protection Regulation (GDPR). Available: <https://gdpr.eu/>
8. U.S. Department of Health & Human Services, HIPAA Guidelines. Available: <https://www.hhs.gov/hipaa/>
9. U.S. Securities and Exchange Commission, Sarbanes-Oxley Act (SOX). Available: <https://www.sec.gov/>